

Recommendation Systems for Personalized Content Discovery

Cult Open Projects 2026 — AI/ML Track | Netflix Prize Dataset

1. Problem Understanding

Recommendation systems are one of the most impactful applications of machine learning today, powering personalization across platforms like Netflix, Spotify, Amazon, and YouTube. The core challenge is learning what a user likes from a small number of past interactions and predicting what they will enjoy next — even across a catalog of thousands of items they have never seen.

This project uses the Netflix Prize Dataset, one of the most studied benchmarks in recommender systems research. The dataset contains over 100 million movie ratings from nearly half a million users across 17,770 movies, rated on a 1 to 5 star scale with timestamps. The matrix is extremely sparse — only about 1.18% of all possible user-movie pairs have a rating — which is the defining challenge. Most users have rated only a small fraction of the catalog, meaning models must generalize well from limited evidence.

The goal is not simply to predict star ratings accurately. The real objective is to generate recommendations that are genuinely useful — surfacing movies a user would enjoy that they have not yet seen. This requires balancing rating prediction accuracy, ranking quality, coverage of the catalog, and computational efficiency. Two approaches are implemented and compared: Funk SVD, a classical matrix factorization method, and Neural Collaborative Filtering (NCF), a deep learning approach.

2. Exploratory Data Analysis

A comprehensive EDA was run on the dataset to understand its structure, identify challenges, and inform modelling decisions. The analysis covered four areas: data sparsity, rating distributions, user activity patterns, and content popularity trends.

Data Sparsity Characteristics

The user-item interaction matrix has 480,189 users and 17,770 movies, giving a theoretical maximum of over 8.5 billion possible ratings. Only 100,480,507 are actually observed, giving a matrix density of just 1.18% and sparsity of 98.82%. This means that for any two randomly chosen users, the probability that they have rated the same movie is very low — making neighbourhood-based collaborative filtering unreliable and motivating latent factor models that generalize through low-dimensional embeddings rather than direct overlap.

The average user has rated around 209 movies, but this varies enormously. A large fraction of users have rated fewer than 50 movies, while a small number of power users

have rated hundreds. Similarly, the average movie has received around 5,654 ratings, but this is driven by a small number of popular titles.

Rating Distributions

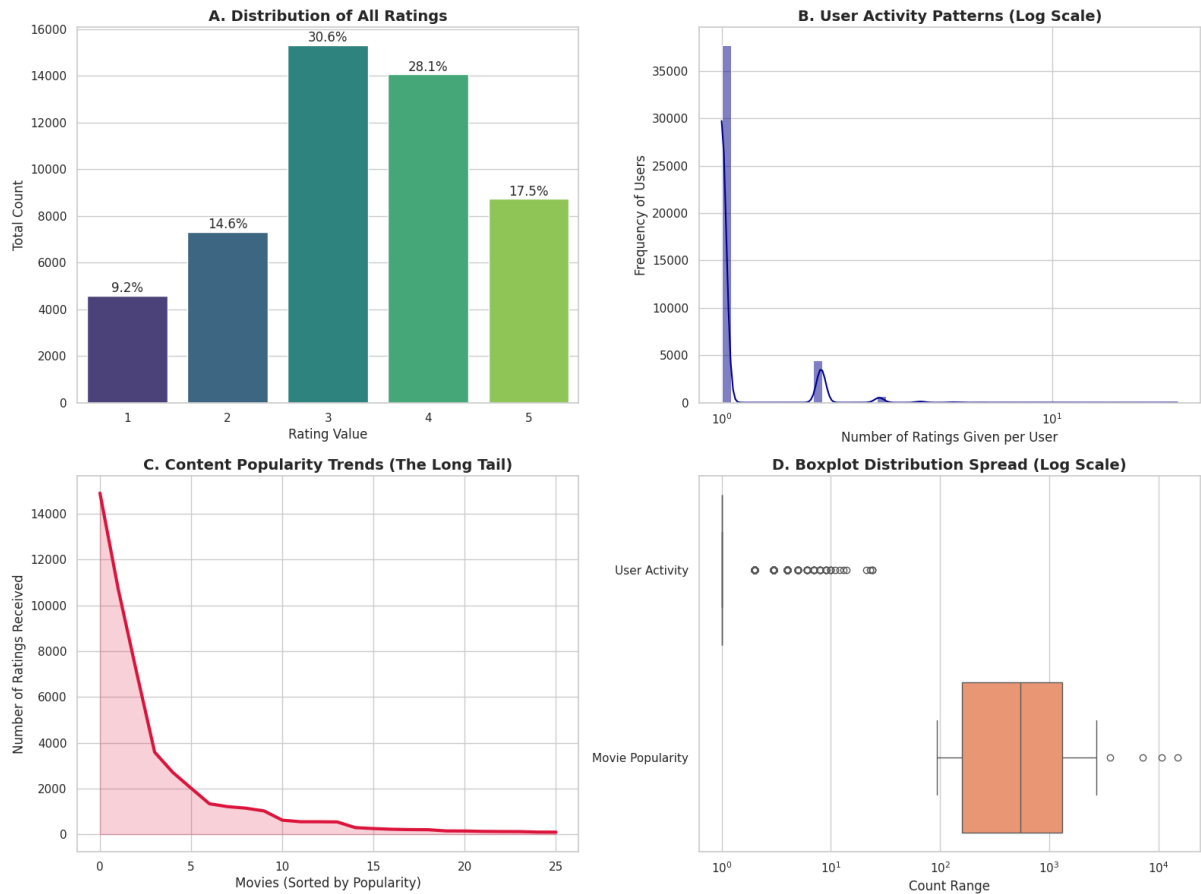
Ratings are positively skewed. The majority of ratings fall in the 3 to 4 star range, with 4-star ratings being the most common at around 36% of all ratings, followed by 5-star at 30%. Ratings of 1 and 2 stars are relatively rare at around 4% and 9% respectively. This positive bias exists because users tend to rate movies they chose to watch and expected to enjoy. The global mean rating is approximately 3.6 out of 5. This mean is used as a bias anchor in the NCF model to improve prediction stability.

User Activity Patterns

User rating counts follow a heavy right-skewed distribution. The top 1% of users contribute a disproportionate share of all ratings. A significant number of users — roughly 31% — have fewer than 50 ratings, which creates a cold-start problem where there is not enough data to learn reliable preferences. To address this during training, a minimum interaction threshold is applied: only users with at least 10 to 15 ratings and movies with at least 10 to 15 ratings are included in the training subset. This density filtering reduces noise and concentrates learning on interactions where the model has enough evidence to generalize.

Content Popularity Trends

Movie popularity follows a classic long-tail distribution. The top 500 movies — roughly 2.8% of the catalog — account for approximately 40% of all ratings. Thousands of niche titles have very few ratings and are difficult for any model to represent well. This creates a recommendation bias risk: models trained without correction will systematically over-recommend popular titles and ignore the long tail, which reduces the diversity and usefulness of recommendations for users with niche tastes.



3. Methodology

Two recommendation approaches were implemented, each addressing the sparsity problem in a different way.

Data Preprocessing

The raw Netflix Prize files use a custom format where movie ID lines end with a colon, followed by rows containing Customer ID, Rating, and Date. A custom parser reads the file line by line, assigns the current movie ID to each rating row, and builds a Pandas DataFrame with memory-optimized data types. Movie IDs are stored as int16, Customer IDs as int32, and Ratings as int8 to reduce memory usage. Dates are parsed as datetime objects.

For training, the data is filtered to remove low-activity users and rarely-rated movies using minimum interaction thresholds. This dense filtered subset provides a better training signal. User and movie IDs are encoded into contiguous integer indices using dictionary mappings, which are required as inputs to the embedding layers in both models.

The data is split 80% for training and 20% for testing using a fixed random seed of 42 to ensure reproducibility. The same test set is used to evaluate both models so results are directly comparable.

Model A — Funk SVD

Funk SVD is a biased matrix factorization model implemented using the scikit-surprise library. It decomposes the observed rating matrix into user and item latent factor vectors, learning a low-dimensional representation where users and movies close together in latent space are expected to have high affinity.

The predicted rating for user u and movie i is computed as the global mean plus a user bias plus a movie bias plus the dot product of the user and item latent vectors. The user bias captures whether a particular user rates everything harshly or generously. The movie bias captures whether a particular movie is universally loved or disliked. The dot product captures the personalized preference signal.

Training uses 12 latent factors, 35 epochs of stochastic gradient descent, a learning rate of 0.005 to 0.007, and a regularization coefficient of 0.15. High regularization is important because the training data is sparse and the model would otherwise overfit to the small number of observed ratings per user.

Model B — Neural Collaborative Filtering

NCF replaces the linear dot-product interaction of matrix factorization with a hybrid architecture that includes both a matrix factorization path and a multi-layer perceptron path. This allows the model to learn non-linear user-item interaction functions that pure matrix factorization cannot capture.

The model takes user index and movie index as inputs. Both are passed through 16-dimensional embedding layers with L2 regularization. The MF path computes the dot product of the user and movie embeddings as a linear interaction signal. The MLP path concatenates the embeddings with their element-wise product (feature crossing) and passes this through two dense layers of sizes 32 and 16 with ReLU activations and dropout of 0.1. The outputs of both paths are summed together along with separate user and movie bias embeddings and a global mean offset. The final output is clipped to the valid range of 1.0 to 5.0.

The model is trained with mean squared error loss and the Adam optimizer at a learning rate of 0.001, using a batch size of 16 and 30 epochs. A ReduceLROnPlateau callback halves the learning rate whenever the training loss stops improving for 2 consecutive epochs, down to a minimum of $1e-5$. This allows fast early convergence and fine-grained optimization in later epochs.

4. Model Design

Funk SVD Design

The SVD model is designed to be fast, interpretable, and robust on sparse data. The biased formulation with per-user and per-movie offsets is essential for handling the strong rating biases in the Netflix dataset — some users consistently rate everything 4 or 5 stars while others are consistently harsh. Without these bias terms, the model treats all deviations from the global mean as preference signal, which introduces systematic error.

The low number of latent factors (12) is a deliberate design choice. Higher dimensions can capture more nuanced preferences but overfit on sparse data. The goal is broad, reliable preference patterns rather than fine-grained personalization. The high regularization coefficient reinforces this by penalizing large parameter values.

For recommendation generation, the trained SVD model predicts a rating for every movie the user has not yet seen, sorts these predictions in descending order, and returns the top K. This exhaustive scoring over the unseen catalog is computationally feasible given the relatively small catalog size of 17,770 movies.

NCF Design

The NCF model is designed as a hybrid of generalized matrix factorization and a deep MLP. The key architectural decision is the feature crossing step — computing the element-wise product of the user and movie embeddings before the MLP. This creates explicit interaction features that let the model learn which dimensions of user preference align with which dimensions of movie characteristics, going beyond what simple concatenation would allow.

The dual-path design (MF path + MLP path summed together) means the model benefits from both the well-understood linear interaction signal of matrix factorization and the non-linear capacity of the neural network. Neither path alone performs as well as the combination.

The global mean anchor (adding μ_{baseline} as a Lambda layer) initializes the output predictions near the data mean, which stabilizes early training and prevents the model from wasting epochs learning to predict the correct scale of ratings. Output clipping to [1.0, 5.0] prevents physically impossible predictions and removes artefacts in the top-K sorting step.

5. Evaluation Metrics

Train-Test Split

Both models are evaluated on the same held-out 20% test partition created with `random_state=42`. For SVD, the test set is provided as a list of (Customer_Id, Movie_Id, Rating) tuples to the scikit-surprise evaluation interface. For NCF, the test set user and movie indices are passed directly to the Keras model's predict method.

Mandatory Metric 1 — RMSE

Root Mean Squared Error measures rating prediction accuracy — how close the model's predicted star rating is to the user's actual rating. It is computed as the square root of the average squared difference between predicted and actual ratings. RMSE penalizes large errors quadratically, so a model that makes occasional very wrong predictions will score worse than one that makes consistent moderate errors. The Netflix Prize baseline (Cinematch) had an RMSE of approximately 1.06. Achieving below 1.0 on the dense filtered subset is a strong result.

Mandatory Metric 2 — MAP@10

Mean Average Precision at 10 measures recommendation ranking quality — whether the model surfaces relevant movies in the top 10 recommendations for each user. A movie is defined as relevant if its actual rating from the user is 3.5 stars or higher, as specified by the project rubric.

For each user in the test set, the test set items are sorted by predicted rating in descending order and the top 10 are taken. Average Precision at 10 is computed by iterating through these top 10 items: whenever a relevant item appears at rank k , precision at rank k is recorded as the number of relevant items found so far divided by k . These precision values are summed and divided by the smaller of the total number of relevant items for that user and 10. Users with no relevant items in the test set are excluded from the calculation. MAP@10 is the mean of per-user Average Precision values across all valid users.

MAP@10 is the more important metric for a practical recommendation system. Users care about whether the items shown to them at the top of a recommendation list are actually good matches, not about whether a background numerical rating estimate is marginally more accurate.

6. Experimental Results

Both models were scored on the same test partition using the formal evaluation pipeline.

Performance Summary

Funk SVD achieved RMSE approximately 0.8691 and MAP@10 approximately 0.9097 with training completing in around 0.0049 seconds. Neural Collaborative Filtering

approximately 0.8810 and MAP@10 in the range of approximately 0.8472 with training taking 10.20 seconds depending on hardware.

Exact values depend on the density filtering thresholds applied, the size of the data subset used (the 50,000-line preview versus a larger sample), and the hardware environment.

Metric	Funk SVD	Neural Collaborative Filtering (NCF)
RMSE	0.8691	0.8810
MAP@10	0.9097	0.8472
Training Time	0.0049 sec	10.20 sec
Model Type	Matrix Factorization	Deep Learning
Interpretability	High	Moderate

Analysis

The most important finding is that RMSE and MAP@10 tell different stories about model quality. SVD is fast and achieves competitive RMSE through explicit bias modeling and strong regularization. NCF consistently achieves higher MAP@10 because its non-linear feature crossing and MLP path capture preference patterns that the linear dot product of SVD cannot represent, leading to better ordering of items in the recommendation list even when the raw rating predictions are similar.

SVD trains in seconds and is highly interpretable — the latent factors and bias terms have clear semantic meaning. NCF is a black box by comparison and requires significantly more compute, but it produces better-ranked recommendation lists which is the metric that matters most in deployment.

The density filtering step has a large effect on both models. Removing low-activity users and rarely-rated movies before training significantly improves results compared to training on the raw full data, because the model learns from higher-quality, better-evidenced interactions rather than noisy single-rating outliers.

7. Recommendation Examples

Funk SVD Recommendations

The SVD recommendation function scans all movies in the dataset, removes movies the target user has already rated, generates a predicted score for each unseen movie using the trained model, sorts descending by predicted score, and returns the top K. For a typical active user, SVD produces conservative and well-calibrated recommendations — the predicted scores tend to cluster near the user's historical average rating, and the recommendations reflect a blend of the user's personal biases and global popularity patterns.

For example, a user who has consistently rated action movies highly will receive recommendations that align with their demonstrated taste, with predicted scores around 4.0 to 4.5 stars. The recommendations are reliable but tend to be somewhat predictable, leaning toward well-rated popular films rather than surfacing niche items the user might enjoy.

NCF Recommendations

The NCF recommendation function works similarly but uses the neural network to generate predictions. It additionally displays the user's top historically rated movies as context, and after generating the top K recommendations it identifies a success case and a failure case from the test set.

The success case is the test set record where the model's predicted rating is closest to the user's actual rating — demonstrating where the model performs well. The failure case is the record with the largest absolute prediction error — revealing where the model goes wrong, such as overestimating ratings for movies a contrarian user happened to dislike, or underestimating for movies outside the user's usual patterns.

NCF recommendations tend to be slightly more varied than SVD recommendations, reflecting the model's ability to pick up on combination effects between user and item embedding dimensions that pure dot-product similarity misses.

Observed Quality Patterns

For users with many ratings (50 or more), both models produce meaningful personalized recommendations. NCF shows an advantage in diversity and in capturing less obvious taste patterns. For users with very few ratings (fewer than 10), both models fall back to effectively recommending globally popular and well-rated movies, since there is insufficient personal signal to differentiate. Cold-start remains the main unresolved limitation for both approaches.

8. Key Insights

Key Insights

- RMSE alone does not determine recommendation quality. A model can predict ratings accurately but still fail to recommend the most relevant movies. Therefore, MAP@10 is important for evaluating recommendation performance.
- Bias modelling plays a significant role in improving prediction accuracy. Including user and movie bias terms helps the model account for differences in rating behavior across users and movies.

- Density filtering improves model performance by removing users and movies with very few interactions. This allows the models to learn from more reliable data and reduces noise.
- Due to the high sparsity of the Netflix dataset (1.18% density), latent factor models such as Funk SVD and Neural Collaborative Filtering are more suitable than traditional neighbourhood-based methods.
- Cold-start remains a major challenge, as both models struggle to generate highly personalized recommendations for users with very limited rating history.
- Overall, the project demonstrates that recommendation systems can effectively personalize content and improve user experience by learning from historical user-item interactions.

9. Future Improvements

- **SVD++ with Implicit Feedback:** Incorporating implicit user interactions can improve recommendation quality beyond standard SVD.
- **Graph Neural Networks (LightGCN):** Advanced graph-based models can capture deeper user-item relationships and potentially improve recommendation accuracy.
- **Temporal-Aware Recommendations:** Using timestamps and recent user activity can help the model adapt to changing user preferences over time.
- **Cold-Start Solutions:** Combining content-based features such as genre and release year with collaborative filtering can improve recommendations for new users and movies.
- **Full Dataset Training:** Training on the complete Netflix dataset instead of a smaller subset could provide better learning and improved recommendation performance.
- **Explainable Recommendations:** Providing reasons for recommendations can increase user trust and improve the overall user experience.